Little Secret PRD

Product overview

Little Secret is a web-based multiplayer social deduction game inspired by Exploding Kittens' Little Secret. Players join a room, receive hidden roles and words, take turns giving clues, vote to eliminate suspicious players, and play until one side wins.

The product is built for lightweight synchronous play on laptops/phones with a single shared room code and real-time state updates.

Objectives

- Deliver a complete, playable 4-8 player game loop in browser.
- Keep game state authoritative on server and resilient to page reloads.
- Provide strong regression protection via unit, interaction, and E2E tests.
- Preserve an intuitive, themed UI including in-app written rules + official rules resources.

Target users

- Groups of friends/family playing in the same room or remotely.
- Hosts who need quick game setup and room sharing.
- Players unfamiliar with rules who need in-context guidance.

Current scope (implemented)

Core gameplay

- Lobby creation/join by room code.
- Role assignment by player count (Good Kitten, Confused Kitten, Spy Pup).
- Full phase flow:
  - `waitingForPlayers`
  - `info`
  - `submitClue`
  - `vote`
  - `revealRole`
  - `gameOver`
- Multi-round progression with correct round increment behavior.
- Voting with tie metadata and server-side random tie resolution.
- Spy guess branch:
  - correct guess -> spy wins immediately
  - wrong guess -> spy eliminated, game continues
- Winner reasons surfaced in GameOver summary.
- Leave-room flow from lobby and from GameOver.

Rules and onboarding

- In-app "How to Play" modal with written rules by default.
- Role distribution table (4-8 players).

- Links to official resources:
  - PDF: `/Little_Secret_US_FLYER.pdf`
  - Video walkthrough
- Rules access during active gameplay via in-game header info button.

Reliability and recovery

- Event log + snapshot persistence interfaces.
- Postgres-backed event store implementation.
- Recovery on service startup from latest snapshots.
- Client rejoin support using persisted `{roomCode, playerID}`.

Testing

- Go domain + service tests for key game logic.
- Vitest unit coverage for frontend helpers and state handling.
- Storybook interaction tests across phase components.
- Playwright journeys covering lobby, rounds, game over, spy branches, and rejoin.

System architecture

Backend

- Go service with WebSocket endpoint at `/ws`.
- In-memory room hub for active sessions.
- Message-driven event handling in `internal/game/service/events.go`.
- Domain rules in `internal/game`.
- Optional persistence path via event log + snapshots.

Frontend

- React + Vite single-page client.
- `GameStateProvider` owns socket connection and state derivation.
- Phase components render according to server-driven phase.
- E2E data attributes exposed for test stability.

Non-goals / deferred

- Secret Weapons system (optional 6+ player rules variant).
- Full printed tie-defense + revote flow (currently simplified to random pick).

Key product requirements

- Players can complete a full game without refresh.
- If a tab reloads, player can rejoin room/phase when session is valid.
- Rules are discoverable before and during gameplay.
- At game end, players can leave room and return to lobby state.
- Core E2E selectors remain stable to avoid test regressions.

Success criteria

- `make test` and `make yarn-test` pass consistently.
- No blockers in 4-player to 8-player main journey.
- GameOver accurately reports winner side and reason.
- Rejoin flow works in mid-game and does not corrupt room state.

Operational notes

- Apply Supabase migration for `ls_game_events` before expecting persistence in live/local DB-backed runs.
- If backend is running stale code, restart service before validating rejoin.